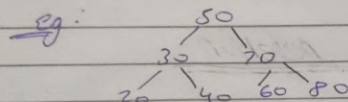


Binary search Tree

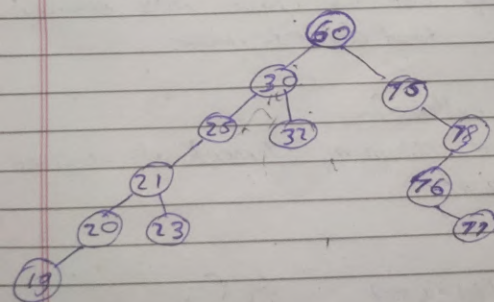
It is a type of binary tree which each node follows rules:

- All values in the left side are less than the node's value
- Values at right are greater than the node's value
- Both left and right subtrees are also binary search trees.



* $\frac{1}{n+1} \sum_{n=0}^{\infty} 2^n C_n$ formula

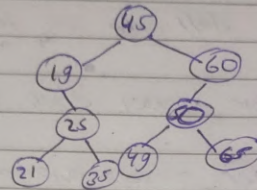
Q Construct BST of 60, 30, 25, 32, 25, 20, 78, 23, 77, 21, 19



* Inorder of BST is in ascending order.

Q. 45, 19, 60, 50, 49, 25, 35, 21, 65

note: order me hi dalna hai



Binary search tree deletion

- There is a no change in BST if we are deleting a leaf node
- when we are deleting an internal node there is a change:

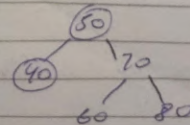
Case 1: LST (left subtree)

if we are selecting lst we choose max element to replace the deleted node.

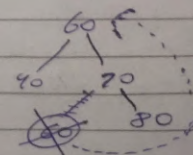
Case 2: RST (right subtree)

we will choose minimum element to replace the deleted element.

Ex. RST deletion



delete(50)



Time Complexity:

	Best	Avg.	Worst
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	"	$O(\log n)$	"
Search	"	"	"

Ques:

* When only Pre/Post order is given and says draw search tree.
 Always know that In-order is in increasing order.

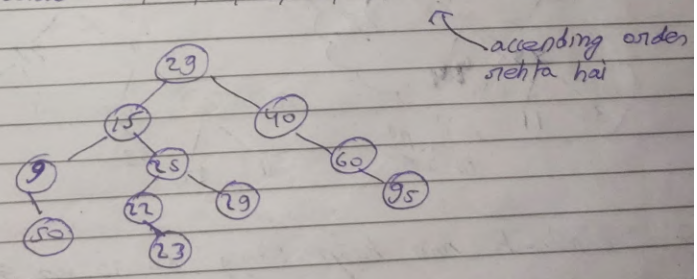
Example:

Post order BSTs: 10, 9, 23, 22, 27, 25, 15, 95, 60, 29

find pre-order

we have,

In-order: 9, 10, 15, 22, 23, 25, 27, 29, 40, 60, 95

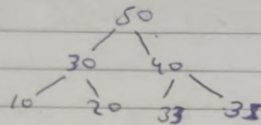


Heap

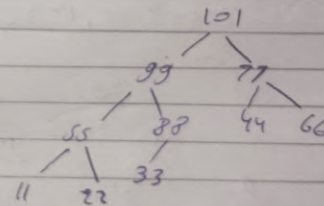
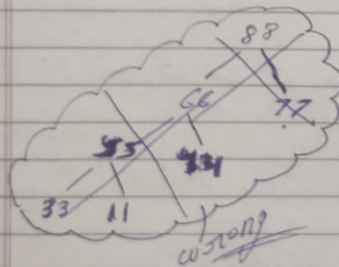
- A heap is a complete binary tree
- Binary heap is of two type:
 1. min heap
 2. max heap.

→ max heap : apne children nodes se bada hoga node at each level

Eg:

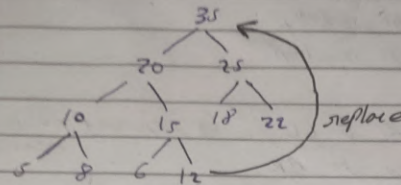


Q. Construct max heap using 44, 33, 77, 11, 55, 88, 101, 22, 99,



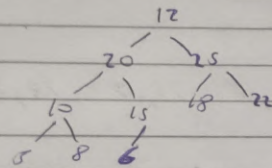
Q. Construct min heap using 60, 30, 40, 90, 80, 100, 20, 50, 70

Extract Max



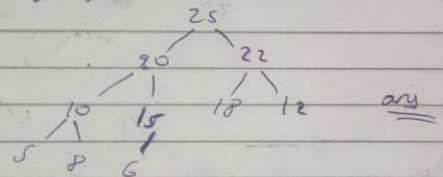
- last i.e. right side
 wala se replace

we get,



Now apply max heap
 properties.

we get,



ans.

Height Balanced Tree : AVL Tree

$$\text{Balance factor} = \text{height of left subtree} - \text{height of right subtree}$$

if,

BF = 0, -1, 1 in range then AVL Tree

if not,

then we make AVL Tree.

* Leaf nodes have balance factor 0.

* AVL Tree is Binary search tree (BST)

⇒ when LL → right rotation

when RR → left rotation

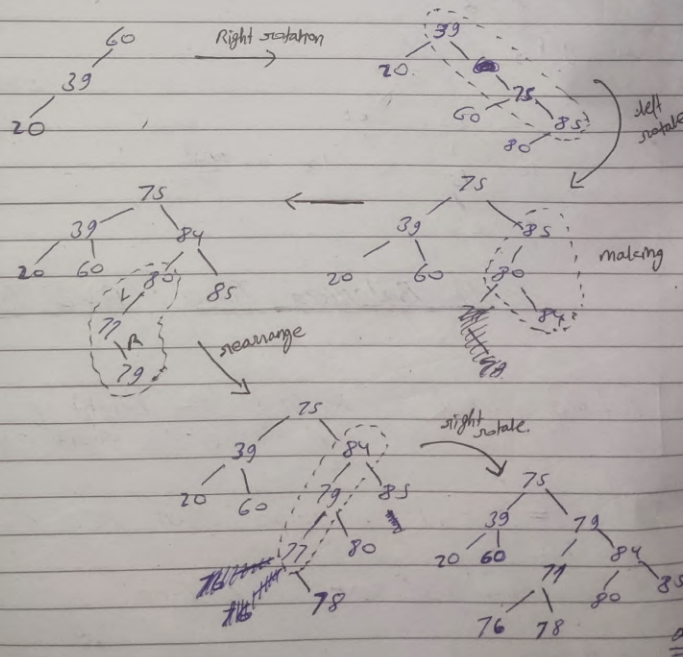
and,

when LR → convert to LL (by replacing position & rearrangement)

when RL → convert to RR

Q Draw AVL Tree:

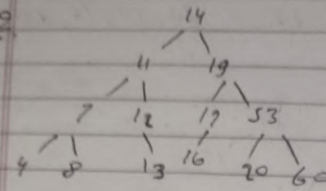
60, 39, 20, 75, 85, 80, 84, 77, 79, 78, 76



Draw AVL Tree

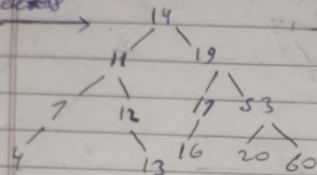


Q9

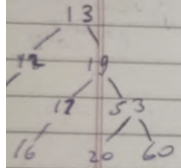
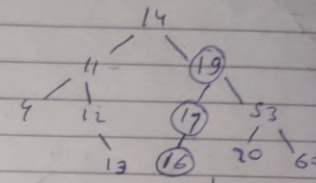


Delete 8, 7, 11, 14, 17 as given in the BST

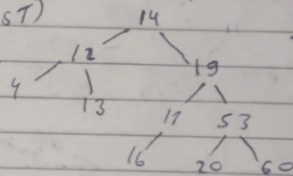
delete 8



delete 7

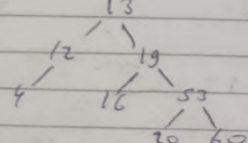


delete (LST)
 replace with maximum



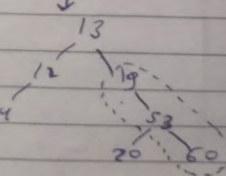
delete 11 (RST)
 replace by minimum element at 11

delete 17

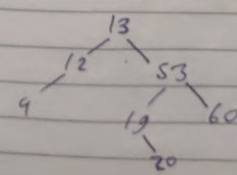


Now, if we delete 16 it will be no longer a BST

delete 16



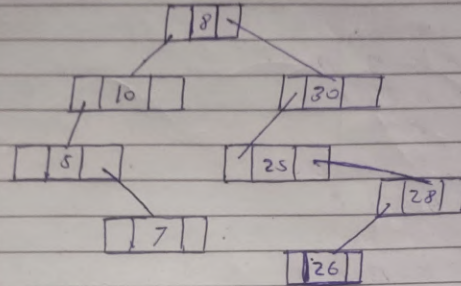
rearrange



* If degree is coming even of all vertices then
it is graph (complete)

Threaded Binary Tree

- single T.B
- double T.B



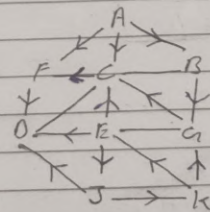
Single T.B

- If right pointer is null then right pointer will contain the address of in-order successor

Double T.B

- Right pointer will contain the address of in-order successor & left pointer will contain address of pre-successor

Breadth First Search



Adjacency List:

A	F, C, B
B	C, G
C	F, E, D
D	C, H
E	C, G, I
F	A
G	B, E, H, I
H	C, D, I
I	G, J
K	J

a) Queue: A b) Queue: A F C B
 origin: ϕ orig: ϕ A A A

c) Queue: A F C B D
 orig: ϕ A A A E

d) Queue: A F C B D
 orig: ϕ A A A E

e) Queue: A F C B D G
 orig: ϕ A A A F B

f) Queue: A F C B D G
 orig: ϕ A A A F B

g) Queue: A F C B D G E
 orig: ϕ A A A F B G

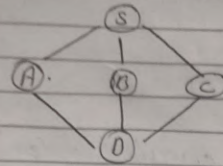
h) Queue: A F C B D G E J
 orig: ϕ A A A F B G E

A F C B D G E J

J is originated from E
 E " " " G
 G " " " B
 B " " " A
 A " " " ϕ

so,
 final path $\rightarrow A \rightarrow B \rightarrow G \rightarrow E \rightarrow J$

Depth first search — uses stack (LIFO)



Adjacency list	
S	A, B, C
A	S, D
B	S, D
C	S, D
D	A, B, C

Algorithm $S \rightarrow A \rightarrow D \rightarrow B \rightarrow C$

Step 1: Visit adjacent unvisited vertex. mark it visited.
 Display it. Push it in a stack

Step 2: If no adjacent vertex found (backtracking will be performed), pop the vertex from stack. It will pop up all the vertices from the stack which do not have adjacent vertices

Step 3: Repeat step 1 & 2 until stack is empty

Differentiate between DFS & BFS

(Spanning Tree)

* $(E - N) + 1$ edges to be remove

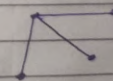
ie



$E = 6$
 $N = 4$

here, $(6 - 4) + 1 = 3$

coming $n-1$ always



* spanning tree me vertices sabhi tree ke andar aur ek dusre dusre se connected rahenge.

* n^{n-2} are no. of possible spanning tree that can be formed from n vertices

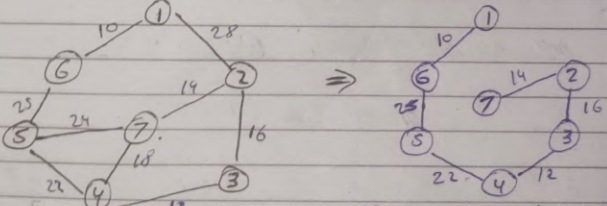
Kruskal's Algo

Algorithm:

MST - KRUSKAL (G, w)

1. $A = \emptyset$
2. for each vertex $v \in G.V$
3. make set cv
4. sort the edges of $G.E$ according to order (or descending)
5. for each edge $(u, v) \in G.E$, taken in non-decreasing order by weight
6. $\text{Find-set}(u) \neq \text{Find-set}(v)$
7. $A = A \cup \{(u, v)\}$
8. $\text{UNION}(u, v)$
9. return A

Prim's Algo



Yes, spanning tree hai

minimum cost $\rightarrow 10 + 22 + 22 + 16 + 14 \Rightarrow 84$

MST - PRIM (G, w, st)

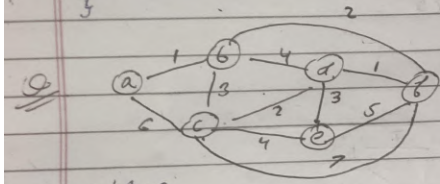
1. for each $v \in G.V$
2. $U.key = \infty$
3. $U.st = \text{NIL}$
4. $\pi.key = 0$
5. $\pi = G.V$

	1	2	3	4	5	6	7
key	∞	∞	∞	∞	∞	∞	∞
π	N	N	N	N	N	N	N

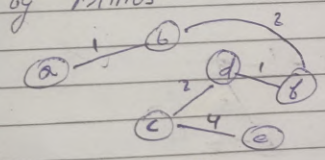
initially

```

6. while  $Q \neq \emptyset$ 
    {
7.    $U = \text{Extract-min}(Q)$ 
8.   for each  $V \in Q$  adj( $U$ )
    {
9.     if  $V \in Q$  and  $w(U, V) < V.\text{key}$ 
10.       $V.\text{key} = w(U, V)$ 
11.    }
    }
    
```

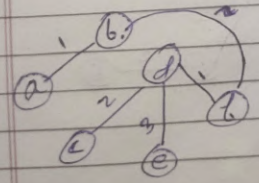
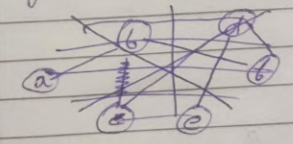


Solution
 by Prim's



minimum cost = $1 + 2 + 1 + 2 + 4$
 $= 10$

⇒ By Kruskal's,



cost ⇒ $1 + 1 + 2 + 2 + 3 = 9$

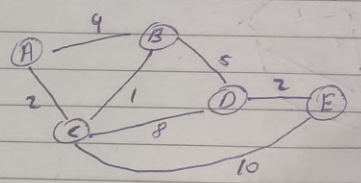
- ✓ $(a,b) = 1$
- ✓ $(d,f) = 1$
- ✓ $(c,d) = 2$
- ✓ $(b,d) = 2$
- ✗ $(b,c) = 3$
- ✓ $(d,e) = 3$
- ✗ $(b,d) = 4$
- ✗ $(d,e) = 5$
- ✗ $(d,a) = 6$
- ✗ $(c,f) = 7$
- ✗ $(c,e) = 4$

Q. $V = \{A, B, C, D, E\}$ Edges & weight

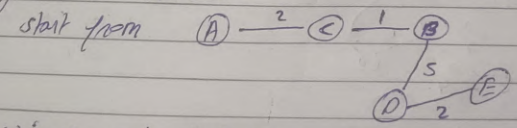
$(A, B) = 4$
 $(A, C) = 2$
 $(B, C) = 1$
 $(B, D) = 5$
 $(C, D) = 8$
 $(C, E) = 10$
 $(D, E) = 2$

find MST a) Prim's start from vertex A
 b) Kruskal's

Solution

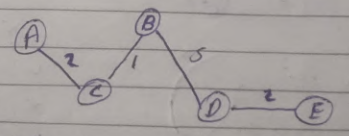


⇒ By Prim's method:



minimum cost $\Rightarrow 2 + 1 + 5 + 2$
 $= 10$

⇒ By Kruskal's method:
 by ascending order,



Cost $\Rightarrow 2 + 1 + 5 + 2$
 $= 10$

Shortest path algorithm

RELAX (u, v, w)

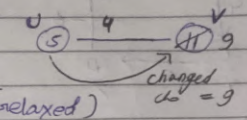
1. If $d[v] > d[u] + w(u, v)$
2. then $d[v] \leftarrow d[u] + w(u, v)$
3. $\pi[v] \leftarrow u$

weight $w(u, v)$

relax
function

here, $11 > 9$

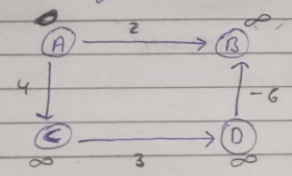
so, $d[v] = 9$ (relaxed)



Bellman Ford

every

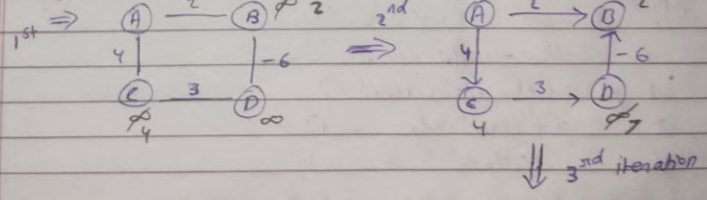
- here edge relax is $(V-1)$ where, V = no. of vertices



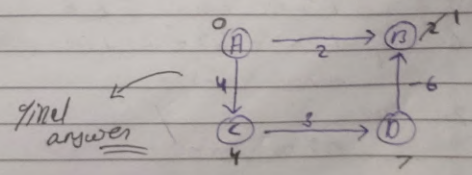
here, we have to apply
relax function $(V-1)$ times
so, $4-1 = 3$ times on each
edges

Given order $\rightarrow (A, B), (A, C), (C, D), (D, B)$

Now,



3rd iteration

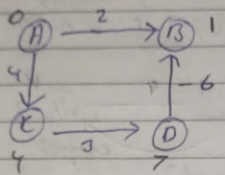


Note

- now To ^{check} ~~find~~ negative weighting cycle in graph

Solution

Ex bar Paise kardo final answer ~~to me~~ relax n app



* Agar koi changes hove
to negative weighted
cycle nhi hai graph

here,

no change occurring, Hence, No -ve wc ^{any}

⇒ Difference b/w Diffn & Bellman

	<u>Diffn</u>	<u>Bellman</u>
1. +ve w	✓	✓
2. -ve w	✓/X	✓
3. -ve wc	X	X

Algorithm:

Bellman Ford (G, w, d)

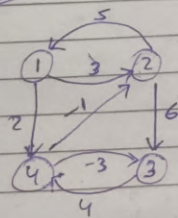
1. initialize - single-source (G, s)
2. for $\lambda = 1$ do ($|V|-1$)
3. for each edge $(u, v) \in G.E$
4. Relax (u, v, w)
5. for each edge $(u, v) \in G.E$
6. if $d[v] > d[u] + w(u, v)$
return False
7. else
return True

* Time complexity $\Rightarrow O(VE)$ ^{general}
 when, if $V = E$ then $O(n^2)$
 else $O(\frac{n \cdot n(n-1)}{2})$
 $= O(n^3)$

Floyd Warshall

- it doesn't detect negative weighting cycle; it works on both directed & undirected graph
- both weights are allowed i.e. -ve & +ve weight
- it also does not give right result on -ve weighted cycle
- $distance[u][v] = \min(distance[u][v], distance[u][k] + distance[k][v])$

Example



$$D^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & 3 & \infty \\ \infty & 0 & 6 & \infty \\ \infty & \infty & 0 & 4 \\ \infty & -1 & -3 & 0 \end{bmatrix} \end{matrix}$$


$$D^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & 3 & \infty \\ 5 & 0 & 6 & 7 \\ \infty & \infty & 0 & 4 \\ \infty & -1 & -3 & 0 \end{bmatrix} \end{matrix}$$

→ write as it is as prev

~~$(2,4) = (2,1) + (1,4)$
 $\infty = 5 + 2$
 $\infty = 7$~~

$(2,3) = (2,1) + (1,3)$
 $6 = 5 + \infty$
 $6 = \infty$
 } 6 is minimum

$$\begin{aligned} (3,1) + (1,2) \\ (3,2) &= \infty + (2,1) \\ \infty &= \infty + 5 \\ \infty &= 3 \end{aligned} \left. \begin{array}{l} \text{3 is} \\ \text{minimum} \end{array} \right\}$$

now,
 D^2 is 2nd row & column as it is

$$D^2 = \begin{bmatrix} 0 & 3 & 9 & 2 \\ 5 & 0 & 6 & 7 \\ \infty & \infty & 0 & 4 \\ 4 & -1 & -3 & 0 \end{bmatrix}$$

and,

$$D^3 = \begin{bmatrix} 0 & 3 & 9 & 2 \\ 5 & 0 & 6 & 7 \\ \infty & \infty & 0 & 4 \\ 4 & -1 & -3 & 0 \end{bmatrix} \quad \text{and, } D^4 = \begin{bmatrix} 0 & 1 & -1 \\ 5 & 0 & 4 \\ 8 & 3 & 0 \\ 4 & -1 & -3 \end{bmatrix}$$

Algorithm: Floyd Warshall (ω)

1. $n = \omega$ rows
2. $D^0 = \omega$
3. for $k=1$ to n — $O(n)$
4. let $D^k = d_{ij}^k$ be a new matrix
5. for $i=1$ to n — $O(n)$
6. for $j=1$ to n — $O(n)$
7. $d_{ij}^k = \min \{ d_{ij}^{k-1}, d_{ik}^{k-1} + d_{kj}^{k-1} \}$
8. return D^n

* Total time complexity $\Rightarrow O(n^3)$

uses memorization
and Tabulation

optimality (sequence of decisions)

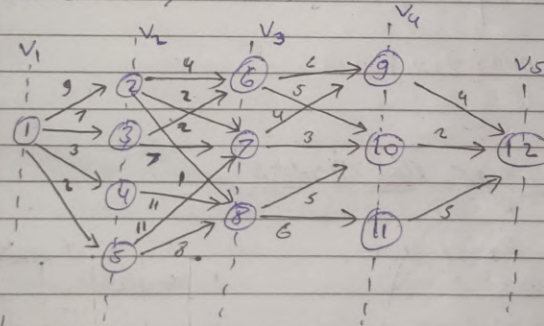
⇒ Dynamic Prog } both are used for
Greedy Approach } optimization solution

- Greedy method & dynamic programming both are used to solve optimization problems
- Dynamic programming is time consuming approach than greedy method.

- * memorization follows top-down approach
- * Tabulation follows Bottom-up approach.

DP Mult stage graph problem

- * Cost (stage, vertex)
- * $c(\text{vertex}, \text{vertex})$



stage V_4
 $\text{Cost}(4, 9) \Rightarrow 4 \rightarrow 12$
 $\text{Cost}(4, 10) = 2 \rightarrow 12$
 $\text{Cost}(4, 11) = 5 \rightarrow 12$

V	1	2	3	4	5	6	7	8	9	10	11	12
Cost						7	5		4	2	5	0
d						10	10		12	12	12	12

Stage V_3 :

$$\text{cost}(3,6) = \min \begin{cases} c(4,9) + \text{cost}(4,9) \\ c(6,10) + \text{cost}(4,10) \end{cases}$$

$$= \min \{ 6+4, 5+2 \}$$

$$= \min \{ 10, 7 \}$$

$$\text{cost}(3,7) = \min \begin{cases} c(7,9) + \text{cost}(9,12) \\ c(7,10) + \text{cost}(10,12) \end{cases}$$

$$= \min \{ 4+4, 3+2 \}$$

$$= \min \{ 8, 5 \}$$

$$\text{cost}(3,8) = \min \begin{cases} c(8,10) + \text{cost}(10,12) \\ c(8,11) + \text{cost}(11,12) \end{cases}$$

$$= \min \{ 5+2, 6+5 \}$$

$$= \min \{ 7, 11 \}$$

Matrix Chain Multiplication

A_1, A_2, A_3, A_4
 $5 \times 4, 4 \times 6, 6 \times 2, 2 \times 7$

$$m[1,1] = m[2,2] = m[3,3] \\ = m[4,4] = 0$$

	1	2	3	4
1	0	120		
2		0	48	
3			0	84
4				0

$$m[1,2]$$

$$A_1 A_2 \Rightarrow 5 \times 6 \times 4 = 120$$

$$m[2,3] \Rightarrow 4 \times 2 \times 6 = 48$$

$$A_2 A_3 \\ 4 \times 6 \quad 6 \times 2$$

$$C[3,4] = \min \{ C[3,3] + C[4,4] + d_2 * d_3 * d_4 \}$$

$$= 0 + 0 + 4 \times 2 \times 5$$

$$= 40$$

Now, three matrices

$$C[1,3] \text{ here, } k=1,2$$

$$A_1 \cdot A_2 \cdot A_3$$

for $k=1$, $C[1,3] = \min \{ C[1,1] + C[2,3] + d_0 * d_1 * d_3 \}$

$$= 0 + 16 + 3 \times 2 \times 2$$

$$= 28$$

for $k=2$, $C[1,3] = \min \{ C[1,2] + C[3,3] + d_0 * d_2 * d_3 \}$

$$= 24 + 0 + 3 \times 4 \times 2$$

$$= 48$$

and, $C[1,3] = \min \{ 28, 48 \} = 28$ ans

$$C[2,4] = \min \begin{cases} k=2, C[2,2] + C[3,4] + d_1 * d_2 * d_4 \\ k=3, C[2,3] + C[4,4] + d_1 * d_3 * d_4 \end{cases}$$

$$= \min \{ 0 + 40 + 5 \times 4 \times 2, 16 + 0 + 2 \times 2 \times 5 \}$$

$$= \min \{ 40 + 40 = 80, 36 \}$$

$$C[2,4] = 36$$

here,

$C[3,2] \rightarrow$ not possible similarly, $[2,4], [4,2], [4,3]$
 $[4,1]$

Now, four matrices,

$$C[1,4] = \begin{cases} k=1, C[1,1] + C[2,4] + d_0 * d_1 * d_4 \\ k=2, C[1,2] + C[3,4] + d_0 * d_2 * d_4 \\ k=3, C[1,3] + C[4,4] + d_0 * d_3 * d_4 \end{cases}$$

$$= \begin{cases} 0 + 36 + 3 \times 2 \times 5 \\ 24 + 40 + 3 \times 4 \times 5 \\ 28 + 0 + 3 \times 2 \times 5 \end{cases}$$

$$C[1,4] = \min \{ 66, 100, 58 \}$$

$$= 58 \text{ ans.}$$

	1	2	3	4
1	0	24	28	58
2		0	16	36
3			0	40
4				0

(minimum values)

	1	2	3	4
1			1	3
2			2	3
3				3
4				

k = matrix

here,

$$58 \Rightarrow C[1,3] + C[4,4] + \dots$$

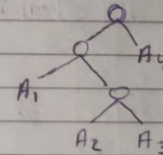
↓

$$(A_1 \cdot A_2 \cdot A_3) \cdot A_4$$

↓ looking at $C[1,3]$

$$(A_1 \cdot (A_2 \cdot A_3)) \cdot A_4$$

Tree solution:



Imp LCS (Longest Common Subsequence)
 (using Dynamic Programming)

string = A B C D

sub-string = A, B, C, D, AB, BC, CD, ABC, BCD, ABCD

subsequence = AB, BC, CD, ABC, BCD

* when we find common subsequence line should not intersect.

$s_1 = a \ b \ a \ d \ a \ c$
 $s_2 = a \ b \ d \ a$

not
* If equal then max from diagonal
ans.

If not equal then both diagonal value added by +1

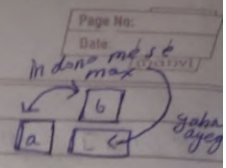


Table:

	0	2	P	2	5	5	Common
0	0	0	0	0	0	0	subsequence → 1/2 5/5
P	0	0	1	0	0	0	
2	0	1	0	2	0	0	if not matching then maximum
P	0	0	2	0	0	0	
5	0	0	0	2	3	3	match karna jo diagonal chote hote karke
2	0	1	0	2	3	0	and,
5	0	0	0	2	3	4	if not matching move
P	0	0	1	2	3	4	

0/1 Knapsack Problem (using Dynamic Programming)

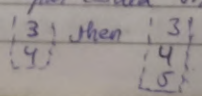
Given, $W_i = \{3, 4, 6, 5\}$ $W = 8$ kg
 $P_i = \{2, 3, 1, 4\}$ $n = 4$
 no. of items

in increasing order

	W →	0	1	2	3	4	5	6	7	8	P _i
P _i ↓	0	0	0	0	0	0	0	0	0	0	
2	3	1	0	0	0	2	2	2	2	2	
3	4	2	0	0	0	2	3	3	3	5	
4	5	3	0	0	0	2	3	4	4	5	6
1	6	4	0	0	0	2	3	4	4	5	6

$P_i \text{ of } 4 > P_i \text{ of } 3$

* Har niche koi value hone pe check karna hoga
 consider karke i.e. 3 then 4



3 and 4 w
 aayega

formula:

$$V[i, w] = \max \{ V[i-1, w], V[i-1, w-w_i] + p_i \}$$

So,

3	$p_i = 2$	max profit $\Rightarrow 6$
5	$p_i = 4$	

Now,

$6 \in 4^{\text{th}} \text{ row} - \text{yes}$
 $6 \in 3^{\text{rd}} \text{ row} - \text{yes}$

} both yes so, $x_4 = 0$

$6 \in 3^{\text{rd}} \text{ row} - \text{yes}$

$6 \in 2^{\text{nd}} \text{ row} - \text{no}$

} $x_3 = 1$
 and, next profit = $(6-4)$
 profit = 2

$2 \in 2^{\text{nd}} \text{ row} - \text{yes}$

$2 \in 1^{\text{st}} \text{ row} - \text{yes}$

} $x_2 = 0$

$2 \in 1^{\text{st}} \text{ row} - \text{yes}$

$2 \in 0^{\text{th}} \text{ row} - \text{no}$

} $x_1 = 1$
 and, next profit = $(2-2) = 0$
 profit

Final answer $\Rightarrow$ According to $w_i = \{3, 4, 5, 5\}$

we get, $x_i = \{1, 0, 1, 0\}$

$x_i = \{1, 0, 0, 0\}$ $\rightarrow$ 5 ka

Get in question $\rightarrow$

$w_i = \{3, 4, 6, 5\}$

so, $x_i = \{1, 0, 0, 1\}$

ans

question ke w_i ke according
 likhenge.

Resource Allocation Problem

- here, 'n' resources are to be allocated to 'm' projects.
- If 'J' resources ($J \leq n$) allocated to 'i' project ($i \leq m$) in such a way the profit $P(i, J)$ and the profit must be maximum.
- we use help of multistage graph.

Example:

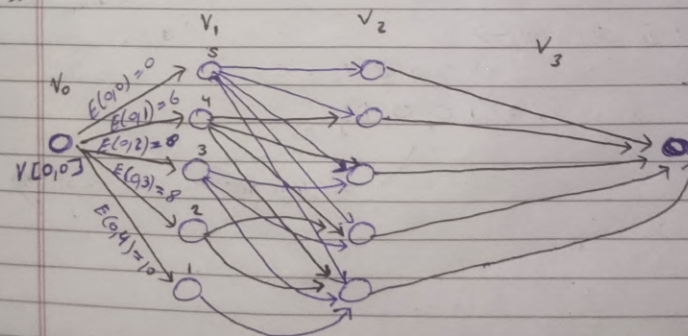
Resources : 4 billion (\$)

Projects : 3

given table

$P_i \backslash j$	0	1	2	3	4
P_0	0	6	8	8	10
P_1	0	5	11	16	17
P_2	0	1	4	5	6

Graph



$V[i, J]$ = profit obtained upto stage 'i' by carrying 'J' resources

$E(m, n) \rightarrow$ profit obtained by spreading 'n' resources on 'm' projects.

Unit-4

Backtracking, Branch & Bound

- It is based on brute-force attack
- It does not work on optimization problem
- Bound condition are given here.
- Backtracking is also a problem solving approach, it uses brute force approach to solve the problem.
- It doesn't work for optimization solution, backtracking follows DFS approach.
- Branch and Bound follows BFS approach

$\Rightarrow$ N-Queen Problem :-

Bound $\rightarrow$ no queens should be on same row, column, diagonal.

* General formula $\rightarrow 1 + \sum_{i=0}^{n-1} \left[\prod_{j=0}^i (n-j) \right]$

Algorithm:

1. N Queens (N)
1. Let rows $[1, \dots, n]$ be a new array filled with zero
 // rows $[c]$ will store the row no. for queen in column c

2. if N -Queens solve (rows, 1, N) = True
3. Print rows
4. return TRUE
5. else
6. Print ("No solution exist")
7. return FALSE

→ N -Queens-solve (rows, col, N)

1. if col > N
2. return TRUE // all queens are safely placed
3. for $\eta = 1$ to N
4. if IS-SAFE (rows, η , col)
5. rows[col] = η // place queen at (η , col)
6. if N -Queens-solve (rows, col+1, N) == TRUE
7. return TRUE
8. rows[col] = 0 // backtracking
9. return FALSE // trigger backtracking to previous col

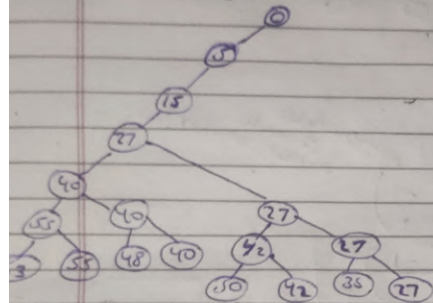
→ IS-SAFE (rows, η , c)

1. for $i = 1$ to $c-1$ // check if another queen is in same row
2. if rows[i] == η
3. return FALSE
4. if $\text{ABS}(\text{rows}[i] - \eta) == \text{ABS}(i - c)$
5. return FALSE
6. return TRUE

Time complexity → $O(N!)$ space complexity: $O(N)$

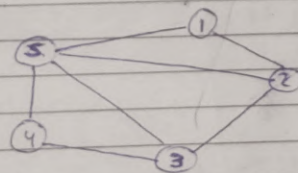
Sum of subsets (solve using Backtracking)

$W[1:6] = \{5, 10, 12, 13, 18, 8\}$
 $n = 6, m = 30$



Time complexity = $O(2^n)$

Graph coloring



$\{R, G, B\}$ No two same colors should be adjacent.

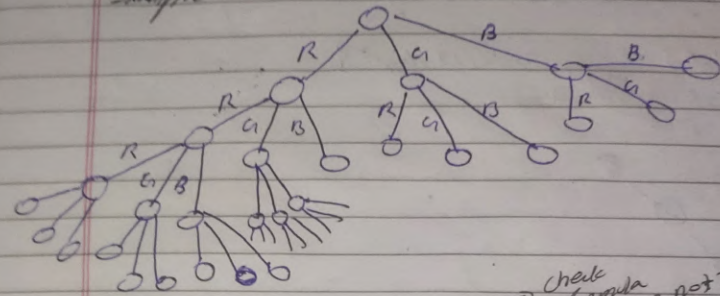
(Adjacency list)

Node	1	2	3	4	5
Color	R	G	R	G	B

1	2, 5
2	1, 5, 3
3	2, 4, 5
4	3, 5
5	1, 2, 3, 4

→ There are more possible combination we find it by state space tree using backtracking.

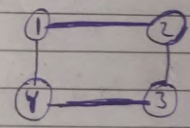
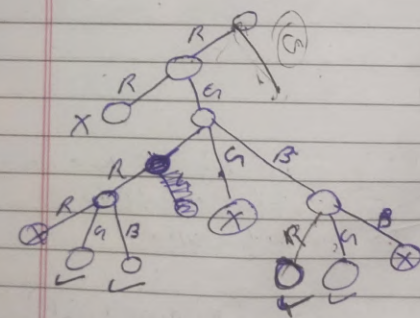
Sample: (without condition)



Total no. of nodes = $a \cdot \left(\frac{a^{n-1} - 1}{a - 1} \right)$
 $= 1 \cdot \left(\frac{3^{4-1} - 1}{3 - 1} \right)$

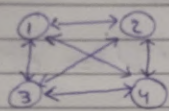
check formula correct or not?

(with condition)



node	1	2	3	4
color	R	G	R	G

Travelling salesman Problem (using D.P)



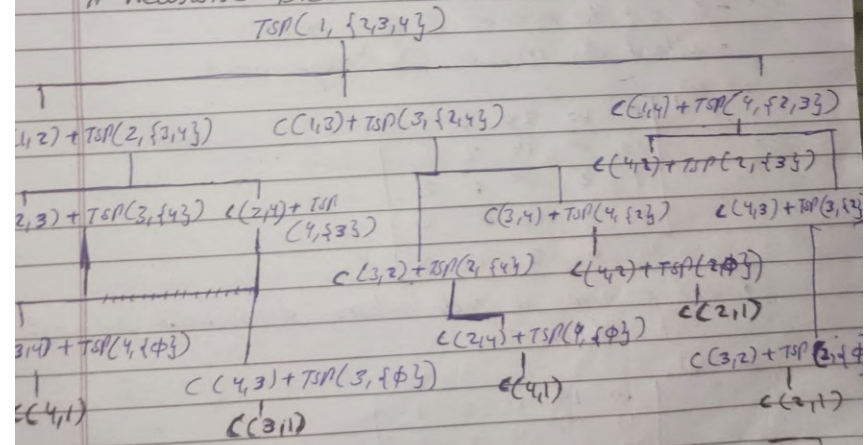
	1	2	3	4
1	0	6	1	3
2	4	0	2	1
3	1	2	0	8
4	3	1	7	0

- Hamiltonian cycle i.e 1-2-4-3-1
- Time complexity by brute force $\rightarrow O(n!) \approx O(n^n)$
 ↑
 without
 D.P

$TSP(1, \{2, 3, 4\})$

$$= \min \begin{cases} C(1, 2) + TSP(2, \{3, 4\}) \\ C(1, 3) + TSP(3, \{2, 4\}) \\ C(1, 4) + TSP(4, \{2, 3\}) \end{cases}$$

Recursive Tree



Note: Formula, $TSP(i, S) = C(i, k) + TSP(k, S - \{i\})$
 where $k \in S$, and when $S = \phi$ then $C(i, 1)$

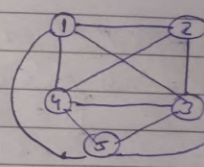
→ How many times TSP function is called?
 ans = 15

→ Time complexity with D.P. = $O((n-1)2^{n-2}) \Rightarrow O(n!)$

TSP (using Backtracking)

- TSP cannot be solved using backtracking so we use here, Branch & Bound.

	1	2	3	4	5
1	∞	20	30	10	11
2	15	∞	16	4	2
3	3	5	∞	2	4
4	19	6	18	∞	3
5	16	4	7	16	∞



Steps

- find out the minimum in every row
- subtract this minimum with every row

we get,

∞	10	20	0	1
13	∞	14	2	0
1	2	∞	0	2
16	3	15	∞	0
12	0	3	12	∞

- find minimum in each column & subtract it from each column.

we get,

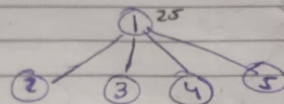
∞	10	17	0	1
12	∞	11	2	0
0	3	∞	0	2
15	3	12	∞	0
11	0	0	12	∞

matrix for node 1

• Find total reduced $\hat{c}_1 = (\text{Total of min of rows} + \text{Total of min of columns})$

$$= 21 + 4$$

$$\hat{c}_1 = 25$$



• finding cost for node 2

① — ②

Row 1 $\leftarrow \infty$

column 2 $\leftarrow \infty$

$C(2,1) \leftarrow \infty$

new $\hat{c}_1 = 0$

Now, for level 1, the reference matrix will be ①

and repeat step 1, 2, 3, 4 for total

we get,

$$\Rightarrow C(1,2) + \text{prev } \hat{c}_1 + \text{new } \hat{c}_1$$

$$= 10 + 25 + 0$$

$$= 35 \quad (\text{for node 2})$$

• finding cost for node ③

① — ③

Row 1 $\leftarrow \infty$

column 3 $\leftarrow \infty$

$C(3,1) \leftarrow \infty$

reference matrix $\rightarrow$ ①

we get,

$$\text{new } \hat{c}_1 = 11$$

$$\Rightarrow C(1,3) + \text{prev } \hat{c}_1 + \text{new } \hat{c}_1$$

$$= 17 + 25 + 11$$

$$= 53 \quad (\text{for node 3})$$

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	11	2	0
3	0	∞	∞	0	2
4	15	∞	12	∞	0
5	11	∞	0	12	∞

on this matrix
(note new matrix banegi as ② after steps)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	12	∞	∞	2	0
3	∞	3	∞	0	2
4	15	3	∞	∞	0
5	11	0	∞	12	∞

similarly; for node 4 $\rightarrow 25$
and, for node 5 $\rightarrow 31$

